eCTF Design Document

Team NEU2

date	version	changes
1/31/2025	1.0	Initial Design Doc
3/9/2025	1.1	Encryption and Authentication of Flash Data
3/17/2025	2.0	Key derivation updates
3/20/2025	3.0	Enhanced IV generation, constant-time HMAC, channel-based key rotation, emergency channel handling

1. Overview

This document provides the initial design specifications in compliance with the Mitre's eCTF 2025 Challenge topology shown below [1]:

It describes the cryptographic algorithms and mechanisms to be used to support encryption and authentication of subscription information and encoded video frames transmitted from the encoder to the embedded MAX78000FTH-based decoder [3]. This design document leverages some of the mechanisms introduced by industry-grade *Real-Time Communications* (RTC) protocols for packetization of encrypted and authenticated medio. The *Internet Engineering Task Force* (IETF) RFC 3711 introduces the *Secure Real-time Transport Protocol* (SRTP) to support secure video transmission over packet networks [2].

In this context, this document describes a solution that is loosely based on the RFC 3711 standard. While RFC3711 provides encryption, authentication, integrity and replay protection, this document updates the cyber suites to comply with current *Federal Information Processing Standards* (FIPS) security requirements. This scheme eliminated *Main-in-the-Middle* (MITM) attacks. In all, these are the features presented in this design specifications:

- Session key derivation based on common crypto secret (as per RFC3711):
    - KV_e: Video Packet Encryption Key

- KV_a: Video Packet Authentication Key
- KS_e Subscription Packet Encryption Key
- KS_a: Subscription Packet Authentication Key
- KF_e: Flash Data Encryption Key
- KF_a: Flash Authentication Key

- *Run-Length Encoding* (RLE) to support the compression of the video frames. This will generate variable size video frames and enlargement of buffers (i.e., UART) to prevent overflow is needed to take into account all cases [4]

- *Advanced Encryption Standard* (AES) 256 *Cypher Block Chaining* (CBC) for packet encryption and data randomization. Because the solution is potentially lossy (there is no actual reliability) *Initialization Vector* (IV) is derived from information known to both encryptor and decryptor. Key size: 32 bytes. Block size: 16 bytes

- *Hash-based Message Authentication Code* (HMAC) based on *Secure Hash Algorithm* (SHA) 256 for packet authentication. Hash size is 32 bytes. Care must be taken to prevent the length extension attack

- *Channel-based key rotation* to derive unique encryption keys for each channel using HMAC-SHA256, providing enhanced security compartmentalization and limiting impact of key compromise

- Enhanced *Initialization Vector* generation using *SHA-256 hashing* of channel IDs with key-derived salt for improved unpredictability and non-linear mapping between channels and IVs

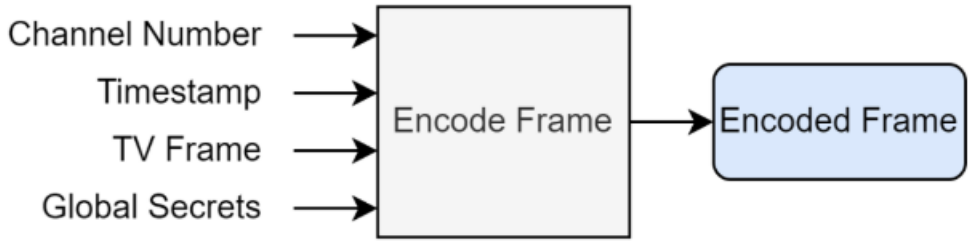
- *Constant-time HMAC verification* for protection against timing side-channel attacks by ensuring validation takes identical time regardless of where differences occur.

- Specialized *emergency channel handling* to ensure channel 0 frames are always properly decoded regardless of subscription status, as required for broadcast functionality.

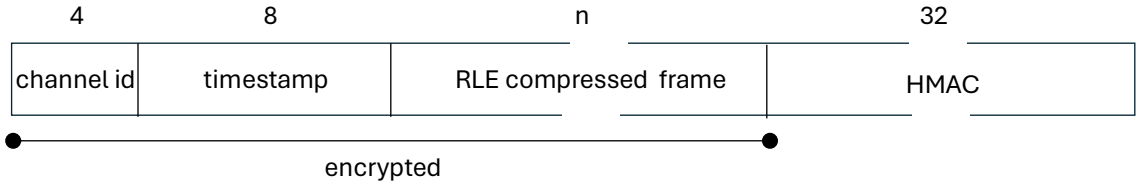
The following sections describe how video and subscription frames are encrypted and authenticated.

2. Video Frame Encryption and Authentication

Each actual video frame consists of a 8x8 ASCII pattern. These 64 bytes are compressed by means of RLE into a variable size compressed frame. Each compressed frame is the payload of a header that includes a 32-bit channel number and a 64-bit timestamp as per the diagram shown below:



The actual encrypted and authenticated packet must comply with the following structure:



It includes the following fields:

- 4-byte channel number: encrypted by AES-256-CBC scheme.
- 8-byte timestamp: encrypted by AES-256-CBC scheme.
- n-byte compressed frame: encrypted by AES-256-CBC scheme. Padding and/or truncation may be needed to comply with the 16-byte encryption block size.
- 32-byte HMAC: Authenticates the encrypted section of the packet. It follows the SHA512 scheme indicated above.

Packing, encryption and authentication must be performed by the encoder (or encryptor) in the following python class member in *design/ectf25_design/encoder.py*

def encode(self, channel: int, frame: bytes, timestamp: int) -> bytes

The pycryptodome package supports all needed encryption (AES-256-CBC) and HMAC capabilities (SHA256). This is the pseudo code:

```
h = HMAC.new(key, digestmod=SHA256)
h.update(message)

cipher = AES.new(key, AES.MODE_CBC, iv)
padded_plaintext = pad(bytes(plaintext), AES.block_size) # Pad to multiple of block size
ciphertext = cipher.encrypt(padded_plaintext)
```

Authentication validation, decryption and unpacking must be performed by the embedded C code in */decoder/src/decoder.c*:

*int decode(pkt_len_t pkt_len, frame_packet_t *new_frame);*

The wolfSSL library supports all needed encryption (AES-256-CBC) and HMAC capabilities (SHA256). This is the pseudo code:

```
result = wc_AesSetKey(&ctx, key, KEY_SIZE, iv, AES_DECRYPTION);
if (result != 0)
{
   return -1;
}

result = wc_AesCbcDecrypt(&ctx, plaintext, ciphertext, len);
if (result != 0)
{
        return result;
}

ret = wc_HmacSetKey(&hmac_ctx, WC_SHA256, key, KEY_SIZE);
if (ret != 0)
{
   return -1;
}

ret = wc_HmacUpdate(&hmac_ctx, message, message_len);
if (ret != 0)
{
   return -1;
}

ret = wc_HmacFinal(&hmac_ctx, computed_hmac);
if (ret != 0)
{
   return -1;
}
```

Note that besides the encryption key, the IV is also a parameter that must be known to both encryptor and decryptor. For video encryption the IV results from a secure hashing of the channel ID with a salt extracted from key material:

```
uint8_t iv\[BLOCK_SIZE\];
uint8_t iv_salt\[8\];
uint8_t channel_hash\[SHA256_DIGEST_SIZE\];
Sha256 sha;

memcpy(iv_salt, kse + (KEY_SIZE - 8), 8);

wc_InitSha256(&sha);
wc_Sha256Update(&sha, (uint8_t*)&channel, sizeof(channel));
wc_Sha256Update(&sha, iv_salt, sizeof(iv_salt));
wc_Sha256Final(&sha, channel_hash);

memcpy(iv, channel_hash, 4);
memcpy(iv + 4, kve + 4, BLOCK_SIZE - 4);
```

This approach ensures IVs are unpredictable and secure, even when reusing keys for different channels.

For enhanced security, we implement channel-based key derivation:

```
void derive_session_key(uint8_t *base_key, uint32_t channel_id, uint8_t *session_key) {
   // Use channel ID to derive a unique key for each channel*
   verify_hmac_sha256(session_key, base_key, (uint8_t*)&channel_id, sizeof(channel_id), true);
}
```

A unique encryption key is derived for each channel, limiting the impact of potential key compromise and preventing cross-channel attacks.

Note that the channel id is sent in the encrypted section of the video packets, so the decoder must loop through the subscribed channels until finding a match. This enhances security protection.

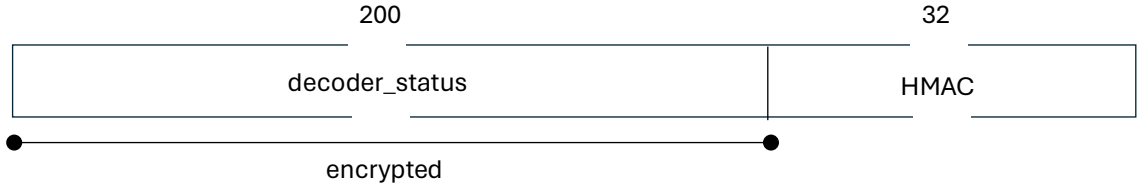
The video decoder implements special handling for emergency channel (channel 0) to ensure these frames are properly decoded regardless of subscription status, as required by the system specifications. The decoder first attempts decryption with the emergency channel key before trying subscribed channels, ensuring emergency broadcasts are always available.

The video decoder will also check for increasing timestamps and timestamps between the ranges of subscription. Also because the timestamp is embedded in the actual encrypted stream, replay protection is ensured.

3. Flash Data Encryption and Authentication

Subscription information is stored in flash memory but to prevent intrusion, the data must be encrypted and authenticated on writing and have authentication validated and then decrypted on reading. The size of the decoder_status structure stored in flash memory is around 1600 bits long.

The encrypted and authenticated packet must comply with the following structure:



It includes the following fields:

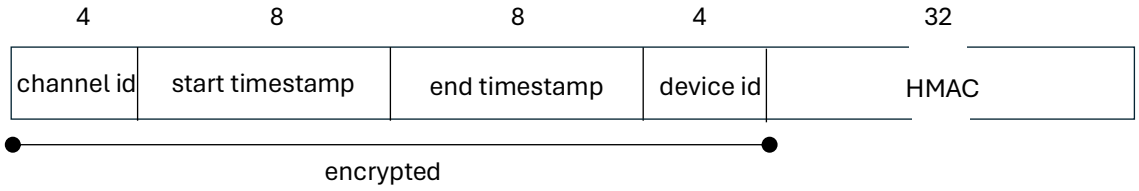
200-byte *decoder_status* structure encrypted by AES-256-CBC scheme. 32-byte *Hash-based Message Authentication Code* (HMAC): Authenticates the encrypted section of the packet. It follows the SHA256 scheme indicated above.

Since it follows the encryption and authentication schemes used for video frame encryption and authentication, refer to Section 2 for API description. For encryption the IV results from using the encryption key:

uint8_t iv[BLOCK_SIZE];
memcpy(iv, kfe, BLOCK_SIZE);

4. Subscription Packet Encryption and Authentication

The subscription is based on the generation of a packet that contains the 32-bit device ID, 64-bit start and end timestamps and the 32-bit channel id. The encrypted and authenticated packet must comply with the following structure:



It includes the following fields:

- 4-byte channel number: encrypted by AES-256-CBC scheme.
- 8-byte start timestamp: encrypted by AES-256-CBC scheme.
- 8-byte end timestamp: encrypted by AES-256-CBC scheme.
- 4-byte device id: encrypted by AES-256-CBC scheme.
- 32-byte *Hash-based Message Authentication Code* (HMAC): Authenticates the encrypted section of the packet. It follows the SHA256 scheme indicated above.

Packing, encryption and authentication must be performed by the encoder (or encryptor) in the following python class member in *design/ectf25_design/gen_subscription.py*:

def gen_subscription(secrets: bytes, device_id: int, start: int, end: int, channel: int) -> bytes

Since it follows the encryption and authentication schemes used for video frame encryption and authentication, refer to Section 2 for API description. For encryption the IV results from using the encryption key:

uint8_t iv[BLOCK_SIZE];
memcpy(iv, kse, BLOCK_SIZE);

During subscription, the subscription decoder ID will be verified to make sure it matches the ID of the actual decoder. Similarly, the channels will be verified to make sure they complied with those specified in the secret.

5. Security Considerations

5.1 Session Key Derivation

Session key derivation will follow the approach specified in Section 4.3.1 of the SRTP RFC3711 specifications. Because the crypto secret is negotiated over the *Session Initialization Protocol*

(SIP) as opposed as provided as a common secret independently in the encoder and decoder some minor modifications are needed. Also, this prevents re-keying which is not possible due to the lack of real-time communication between encoder/encryptor and decoder/decryptor.

To simplify the design, the secret is a 96-byte random string generated as follows:

```
secrets = {
    "channels": channels,
    "some_secrets": os.urandom(48).hex(),
}
```

These secrets are propagated down to the encoder, gen_subscription and decoder modules and keys are generated as HMAC codes of the secret with keys based on segments of the actual secret as follows:

```
kve = hmac_sha256(array, array[0:32])
kse = hmac_sha256(array, array[5:37])
kfe = hmac_sha256(array, array[10:42])

kva = hmac_sha256(array, array[15:47])
ksa = hmac_sha256(array, array[20:52])
kfa = hmac_sha256(array, array[25:57])
```

For example, if the secret is:

581604de4230ac1683a944b2af8a3bc2791cd58837bf7d6cfdc2291df5d6ba4319cdd23b50be31600b6a3284c5c1f4fc

KS_e is obtained

From generating an HMAC SHA256 from

hmac_sha256('581604de4230ac1683a944b2af8a3bc2791cd58837bf7d6cfdc2291df5d6ba4319cdd23b50be31600b6a3284c5c1f4fc', '80078acd9e219fca0edcb83e0da92962')

This derivation is dynamically done on the transmitter and receiver.

5.2 Packet Encryption

Packet encryption is based on AES-256-CBC encryption where the IV is computed based on the channel id and encryption key itself for video packets and on the keys themselves for subscription and flash data.

5.3 Packet Authentication

Packet authentication is based on the generation of a 32-byte HMAC that is computing from performing SHA256 hashing on the data structure that combines the actual encrypted packet data and the 32-byte session authentication key.

5.4 Channel – Based Key Rotation

To enhance security and limit the impact of key compromise, we've implemented channel-based key rotation. For each channel, a unique encryption key is derived from the master key using HMAC-SHA256:

```
void derive_session_key(uint8_t *base_key, uint32_t channel_id, uint8_t *session_key) {
verify_hmac_sha256(session_key, base_key, (uint8_t*)&channel_id, sizeof(channel_id), true);
}
```

5.5 Enhanced IV Generation

This implementation uses SHA-256 hashing of the channel ID with a salt extracted from existing key material:

```
// Extract salt from key material
memcpy(iv_salt, kse + (KEY_SIZE - 8), 8);

 // Hash the channel with the salt* *wc_InitSha256(&sha);
wc_Sha256Update(&sha, (uint8_t*)&channel, sizeof(channel));
wc_Sha256Update(&sha, iv_salt, sizeof(iv_salt));
wc_Sha256Final(&sha, channel_hash);

// Use hash output for IV
memcpy(iv, channel_hash, 4);
memcpy(iv + 4, kve + 4, BLOCK_SIZE - 4);
```

5.6 Constant – Time HMAC Verification

To prevent timing side-channel attacks that could leak information about valid HMACs, we've implemented constant-time comparison:

```
static int constant_time_memcmp(const void* a, const void* b, size_t size) {
        const unsigned char* a_ptr = (const unsigned char*)a;
        const unsigned char* b_ptr = (const unsigned char*)b;
        unsigned char result = 0;
    for (size_t i = 0; i < size; i++) {
         result |= a_ptr[i] ^ b_ptr[i];
      }

        return result;
}
```

5.7 Emergency Channel Handling

The emergency broadcast channel (channel 0) has special requirements - it must always be decodable regardless of subscription status. We've implemented specialized handling to ensure emergency frames are properly processed:

```
// First check if it's the emergency channel
if (emergency_channel == frame->channel) {
        // Process emergency channel with special timestamp handling
```

```
        start_timestamp = 0;
        end_timestamp = UINT64_MAX;
        found = true;
}
```

6. Video Frame Compression

In addition to encryption and authentication, the system will also include a "video compression" stage where the encoder and decoder will rely on *Run-Length Encoding* (RLE) to respectively compress and decompress the 8x8 ASCII video frames during preprocessing. The RLE functionality will be written from scratch without needing external libraries. Compression will be performed before encryption and authentication. Decompression will be performed after authentication validation and decryption.

7. Compliance with Security Requirements

While the sections above cover the functional requirements, the compliance with specific security requirements are described in this section.

7.1 Security Requirement 1

An attacker should not be able to decode TV frames without a Decoder that has a valid, active subscription to that channel.

This will be implemented in the aforementioned device decoder function, where incoming video packets after authentication validation and decryption will be checked to make sure that the channel id is one in the subscription list. Video frames associated with channel 0 will bypass this checking.

7.2 Security Requirement 2

The Decoder should only decode valid TV frames generated by the Satellite System the Decoder was provisioned for.

This will be enforced by supporting authentication by means of HMAC tags in all packets. This not only applies to TV frames but also to subscription packets.

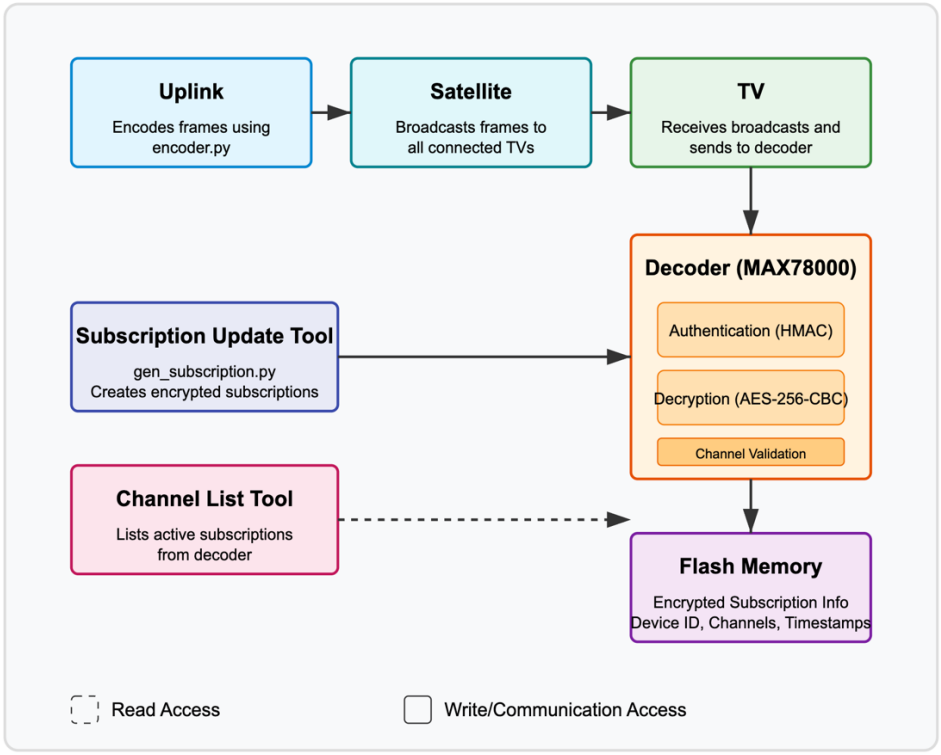
7.3 Security Requirement 3

The Decoder should only decode frames with strictly monotonically increasing timestamps.

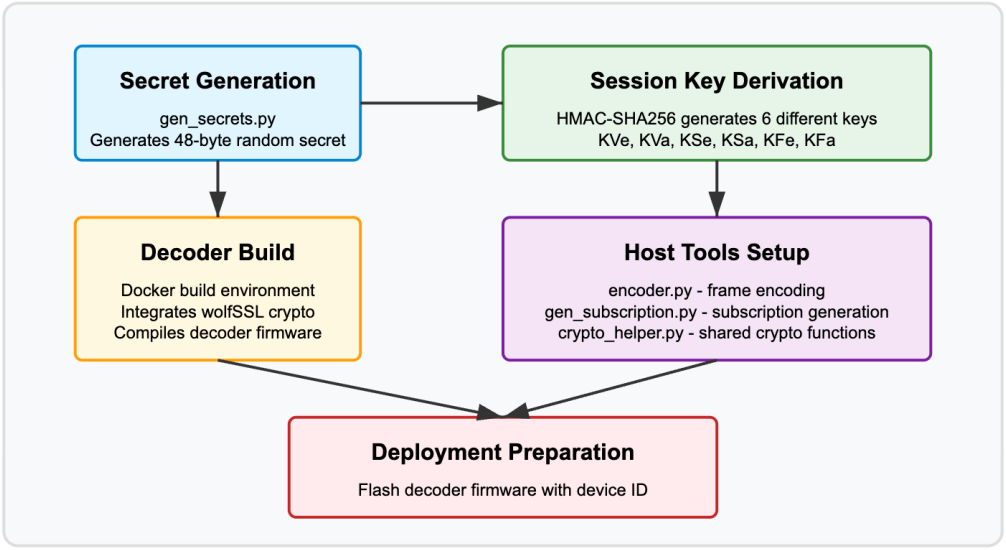
As for requirement 1, this will be implemented in the device decoder function, where incoming video packets after authentication validation and decryption will be checked to make sure that the timestamp complies with this requirement. A basic playout buffer scheme (similar to that used in RTC system) would work in this case. Note that in combination with the security considerations presented in this document, replay protection will be also implemented.

8. System Diagrams

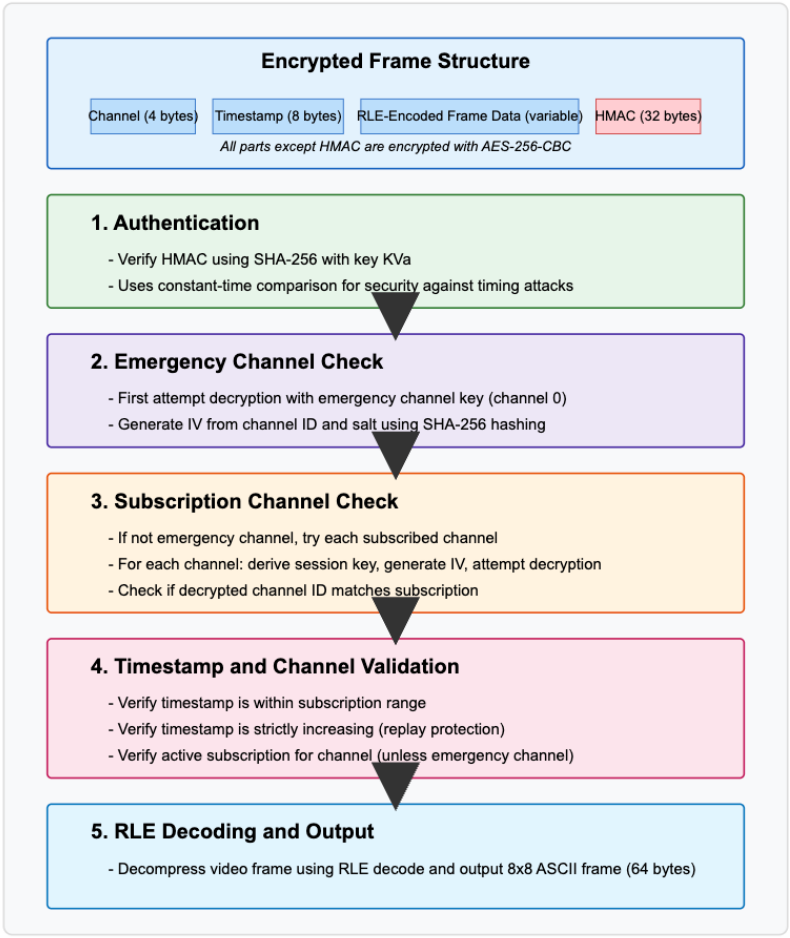
Runtime System Architecture



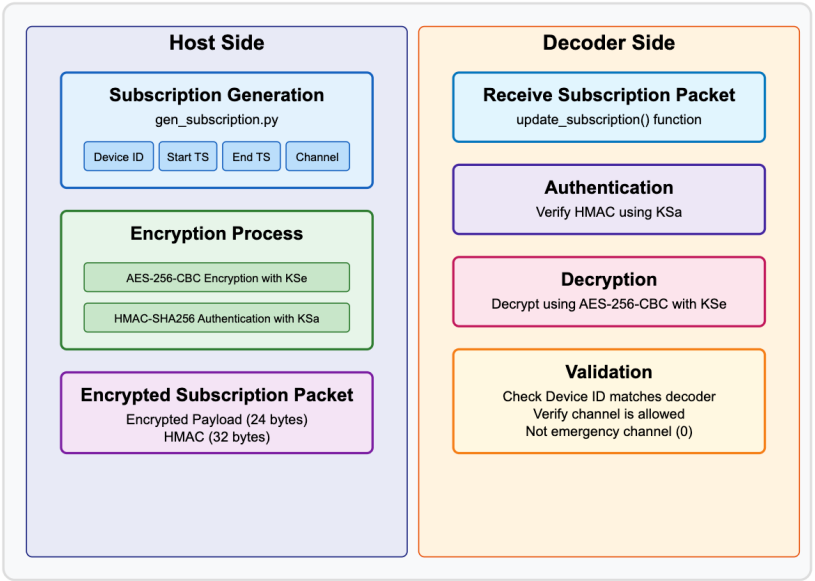
Compile Time System Flow



TV Frame Decoding Process



Subscription Update Process



9. References

1. Mitre's eCTF Rules, https://ectfmitre.gitlab.io/ectf-website/index.html
2. IETF RFC3711, The Secure Real-time Transport Protocol (SRTP), https://ectfmitre.gitlab.io/ectf-website/index.html
3. Analog Devices, MAX78000FTHR, https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/max78000fthr.html
4. Results of a prototype television bandwidth compression scheme, Robinson, A. H.; Cherry, C. (1967)